Architecture — Clinical Co-Pilot

Status: design locked, build in progress

Date: 2026-04-27

1. Executive summary

A multi-turn conversational agent that helps a **hospitalist physician** round on 15–20 inpatients per shift. Reads the patient's electronic chart (OpenEMR via FHIR R4), synthesizes the parts the doctor cares about, and answers in natural language with **every claim cited back to a specific chart record**. The system is read-only over real EHR data, designed for the "hospital CTO bar" of production scrutiny: standards-compliant API, HIPAA-aware auditing, role-based authorization, observability from day one.

The headline differentiator versus a generic medical chatbot is **architectural verification**: the LLM cannot state a fact that wasn't returned by a tool call in the same conversation. Hallucinations don't get prevented at the model level (where they cannot be guaranteed); they're rejected at the system level by a structural validator.

2. System context

Stakeholder	Role
Hospitalist physician	Primary user. Carries 15–20 inpatients. Uses the agent at three points in the shift: morning pre-round, midday med safety check, evening sign-out.
Nurse / resident	Secondary users (read-only access, scoped to their assigned patients).
OpenEMR	The EHR system of record. The agent reads from it via FHIR; it never writes back in v1.
Hospital IT / CTO	The "are we comfortable deploying this" approver. Cares about: standards (FHIR), security (OAuth2, audit log), failure modes (cited claims, refusal-to-confabulate), and scale (300 concurrent users, 500-bed hospital).

3. High-level architecture

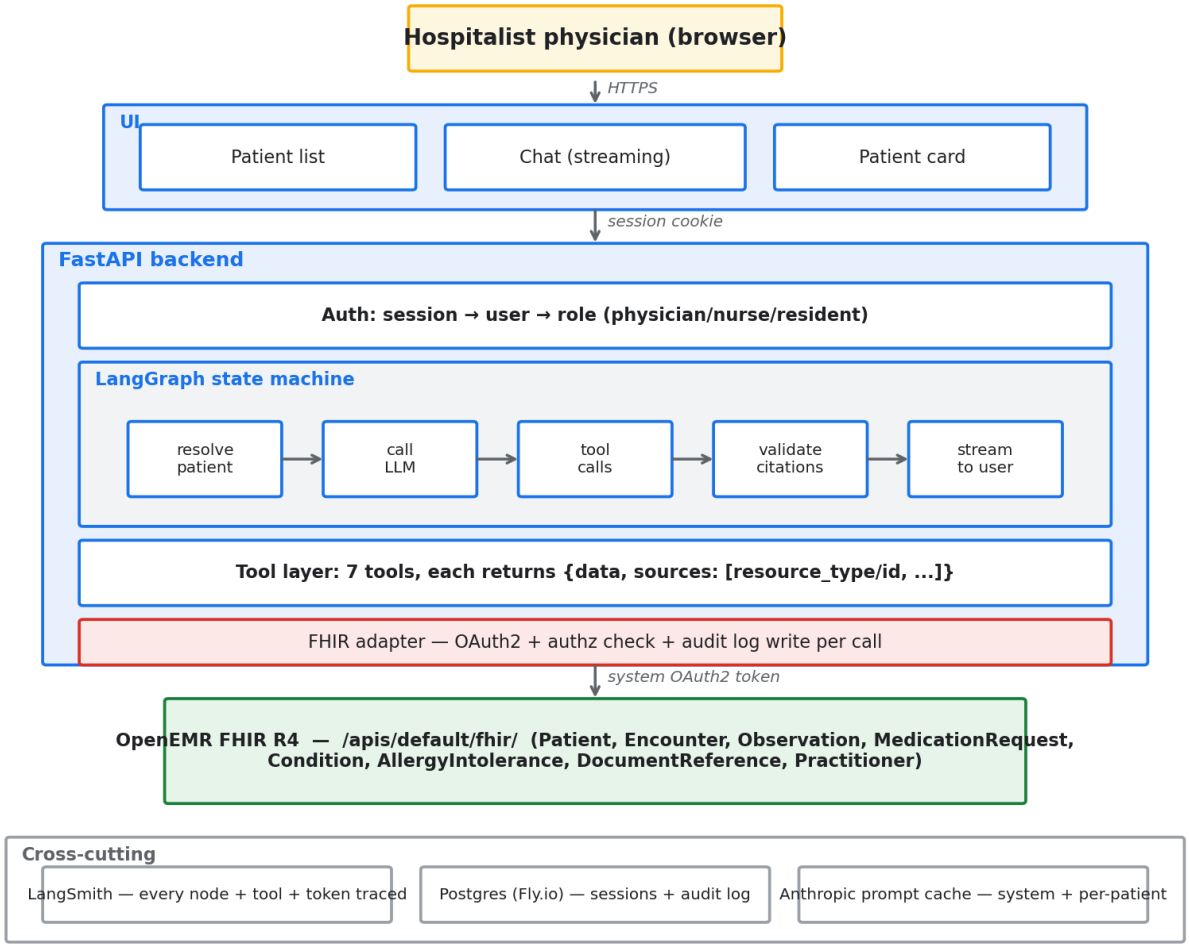
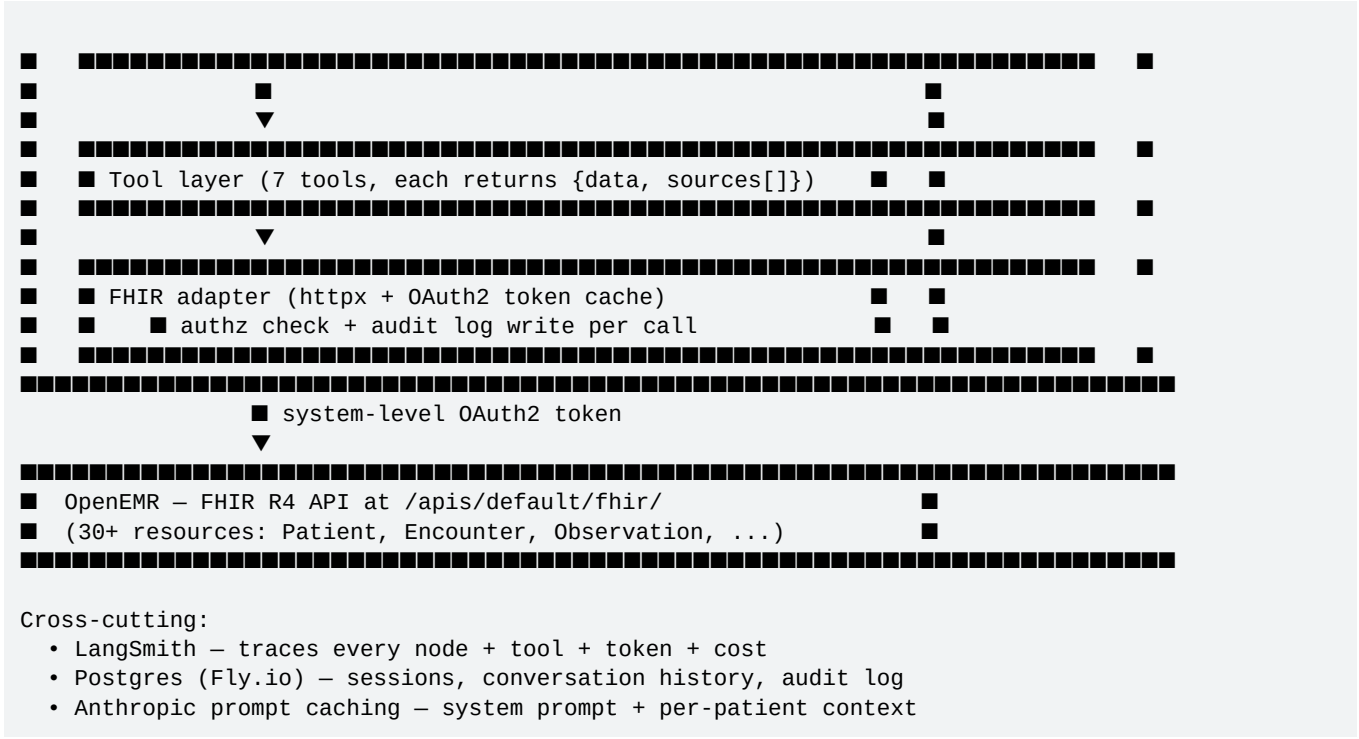


Figure 1 — System architecture overview



4. Layer walkthrough

4.1 Data layer — OpenEMR FHIR R4

• **Why FHIR over the legacy REST API:** FHIR is what real hospital integrations use (Cerner, Epic, Allscripts all expose it). "We use FHIR" is a defensible answer at the CTO bar; "we query MySQL directly" is not.

• **Resources consumed:** Patient, Encounter, Observation (labs + vitals), MedicationRequest, Condition (problem list), AllergyIntolerance, DocumentReference (notes), Practitioner.

• **Auth:** OAuth2 **client_credentials grant** with `system/*` scopes. The agent backend registers once as a confidential client and gets system-level tokens — no per-user OAuth dance, no user redirects.

• Tradeoff: this gives the agent broad chart access, so authorization moves to *our* layer (see 4.3). The alternative — SMART-on-FHIR user OAuth — pushes auth onto OpenEMR but adds a redirect-and-consent flow per session, which is wrong for a backend agent.

4.2 Application layer

• **Backend:** FastAPI (Python). Lightweight, async-native, type-safe.

• **Agent orchestration: LangGraph** (state machine). Chosen over the older `AgentExecutor` because we need an explicit `validate_citations` node between the LLM's output and the user — LangGraph makes intermediate gating natural; AgentExecutor hides it.

• **LLM cascade:**

• Default: **Claude Sonnet 4.6** (fast, ~$3/$15 per MTok). Handles 90% of turns.

• Use Case B (med safety): escalates to **Claude Opus 4.7** for harder multi-fact reasoning.

• Sub-tasks (parameter extraction, classification): **Claude Haiku 4.5** for speed and cost.

• **Tools:** 7 Python functions wrapped with LangChain's `@tool`. Every tool returns `{data, sources: [resource_type/id, ...]}`. The `sources` list is the citation primitive used by the validator.

4.3 Authorization, audit, HIPAA

• **Authorization at the tool layer.** Every tool implicitly takes the caller's `user_id`. The first thing each tool does is call `list_my_patients(user_id)` (or check membership in it) to confirm the requested patient is in scope. If not, the tool returns an error the LLM can handle gracefully ("you don't have access to that record") rather than data leaking.

• **Audit log.** Every tool invocation writes one row to a Postgres `audit_events` table (append-only): `(timestamp, user_id, session_id, tool, args, patient_id, sources_returned)`. This is what HIPAA actually requires — proof of who saw what, when.

• **PHI handling.** Demo data only for the sprint. In production: BAA with the LLM provider (Anthropic offers BAAs via AWS Bedrock); secrets in Fly.io's vault, not env files; TLS everywhere.

4.4 Verification — the differentiator

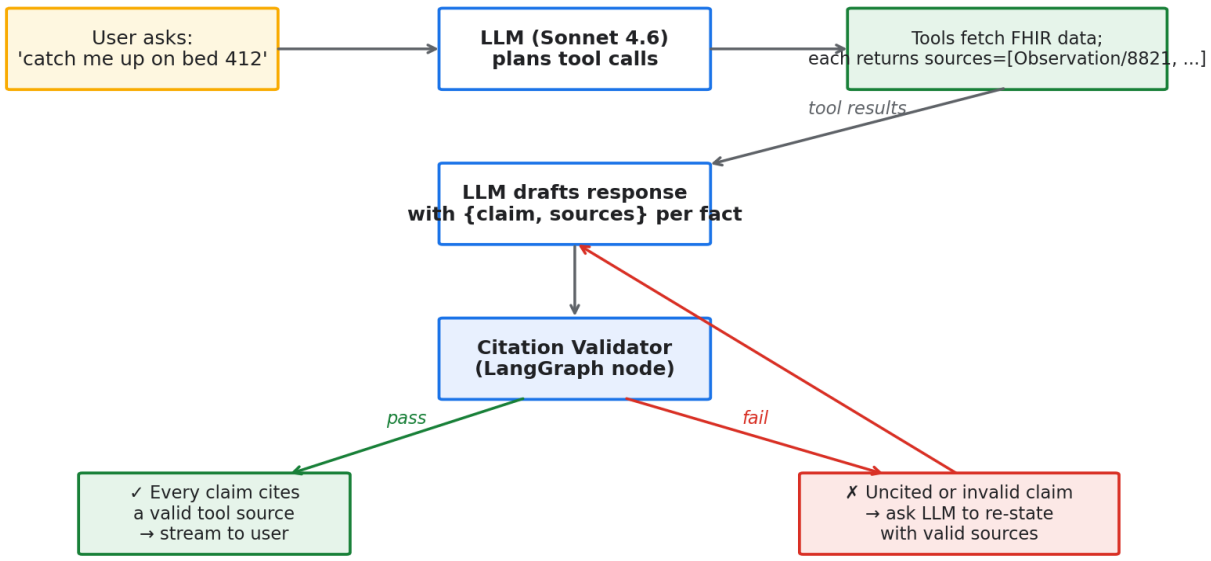


Figure 2 — Verification flow with citation validator

Three layered defenses:

1. **Tool-output-as-source-of-truth (architectural).** The LLM cannot fetch chart data on its own. Every fact in a response originates from a tool call result.

2. **Structured-output-with-citations (prompt-level).** The LLM is required to emit a structured response where each claim carries a `source id`. Example:

```
{"claim": "Cr is 2.1, up from 1.4 yesterday",
 "sources": ["Observation/8821", "Observation/8654"]}
```

3. **Citation validator node (LangGraph).** Before the response goes to the user, a deterministic node checks: every `source_id` in the response must appear in the `sources` list of a tool call from this conversation. If a claim is uncited or cites a nonexistent resource, the response is rejected and the LLM is asked to retry (`messages.append("Citation X/123 not found in tool outputs. Re-state with valid sources.")`).

4. **Post-hoc fact-checker (Use Case B only).** A second LLM call independently verifies that each cited claim is actually supported by the cited resource's content. Slow and expensive, so reserved for medication safety (where being wrong has the highest cost).

Together these guarantee that **a well-behaved agent cannot lie about chart contents** — and a misbehaving one is caught structurally, not heuristically.

4.5 Observability

• **LangSmith** traces every conversation as a tree: user input → LLM calls → tool calls → validator → response. Every node carries latency and cost.

• The same LangSmith project hosts the **eval suite** — a YAML dataset of scenarios per use case, scored on factual accuracy, citation coverage, refusal correctness, and latency. Runs on every commit.

5. Latency model

Streaming is the trick: total response time matters less than time-to-first-word.

Use Case	Time-to-first-word target	Total response target	Reason
A — Pre-round summary	< 2s	< 8s	Doctor is in a hallway between rooms
B — Med safety check	< 2s	< 15s	Decision-grade; willing to wait
C — Sign-out drafting	< 2s	< 60s for 18 patients (stream per patient)	Long output, perceived speed = first patient draft

Performance levers: streaming responses, parallel tool calls (fetch labs and meds simultaneously), Anthropic prompt caching for the system prompt and per-patient context (~90% input cost reduction on repeated turns).

6. Cost model — first-pass projection

Assumptions per session (conservative):

• 4 turns/session, 800 input + 400 output tokens per turn (after caching)

• 70% Sonnet, 25% Haiku, 5% Opus mix

Scale	Sessions/mo	LLM cost/mo (USD)	Infra (Fly + Postgres + LangSmith)	Total
100 users	~600	~$10	~$30	**~$40**
1K users	~6K	~$100	~$60	**~$160**
10K users	~60K	~$1,000	~$300	**~$1,300**

Scale	Sessions/mo	LLM cost/mo (USD)	Infra (Fly + Postgres + LangSmith)	Total
100K users	~600K	~$10,000	~$2,000 (need Postgres scale-out, vector store, dedicated FHIR proxy)	**~$12,000**

Per-physician marginal cost at the 1K-user tier is ~$0.16/month — well under the price of any clinical SaaS.

7. Tradeoffs and alternatives considered

Decision	Chose	Rejected	Why
LLM provider	Anthropic Claude	OpenAI, Google	Best at "refuse to claim what you can't cite" — the exact behavior we cannot tolerate getting wrong. BAA available via Bedrock.
Agent framework	LangChain + LangGraph	Raw Anthropic SDK	Native LangSmith integration, explicit state machine, more documentation for a beginner team. Costs us ~150 lines of abstraction.
FHIR interface	FHIR R4 client_credentials	Direct MySQL queries	Standards-compliant, hospital-defensible, decoupled from OpenEMR's internal schema.
Verification	Structural (tool source-of-truth + validator)	Just prompt-level	Prompts are not enforcement. A model that gets cleverer can still confabulate; a validator cannot be talked out of its check.

Decision	Chose	Rejected	Why
Demo data	Synthea	OpenEMR's 3 built-in patients	Need realistic inpatient data with multi-day notes, lab trends, med changes for hospitalist scenarios.
Hosting	Fly.io	AWS, Vercel	Fast deploy (one command), managed Postgres, edge-distributed; defensible to a CTO because it's Docker-native (no proprietary lock-in).
App database	Postgres	SQLite, MongoDB	Postgres handles JSON well (`jsonb` for tool args), ACID for the audit log, scales horizontally on Fly. SQLite breaks at >1 instance; Mongo wins nothing for our shape.

8. Production-readiness gaps (honest)

This is what would still be required to ship this to a real hospital:

- **BAA with LLM provider** — for the sprint demo we use Anthropic direct (no PHI); production routes through AWS Bedrock with a signed BAA.
- **Real authentication** — current plan is OpenEMR session passthrough; a hospital deployment would integrate with their SSO (SAML/OIDC).
- **Encryption at rest** in our Postgres for conversation history and audit log (Fly.io managed Postgres has this; configuration check needed).
- **Drug interaction database** — for Use Case B, we'd license First Databank (FDB) or RxNorm-DDI rather than relying on the LLM's training knowledge of interactions.
- **Disaster recovery** — backup/restore SLA, multi-region failover, RPO/RTO targets.
- **Clinical validation** — IRB review, physician evaluation panel before live use.
- **The chart UI** — we ship a prototype; production needs a clinical UX review.

9. Build sequencing

Sprint gate	Date	Scope
Architecture defense	2026-04-27 evening	This doc + presearch.md + verbal walkthrough
MVP	2026-04-28 (Tue) 11:59 PM CT	Use Case A end-to-end, deployed on Fly.io, 3–5 min demo, AUDIT.md
Early submission	2026-04-30 (Thu) 11:59 PM CT	+ Use Case B + LangSmith traces visible + eval dataset green
Final	2026-05-03 (Sun)	+ Use Case C + cost analysis + social post + production polish

Use Case A ships first because it's the broadest "feels like the agent works" demo and exercises the full architecture (every tool, the verification node, streaming, citations).

10. The defense — what to expect

Likely questions and the short answer for each:

Question	Answer
"Why a chat agent and not a dashboard?"	Doctors ask follow-ups ("show me the trend"; "what's he on that affects K?"). A dashboard can't anticipate the thread.
"How do you stop hallucinations?"	Architectural: every claim must trace to a tool result. A validator rejects the response if a citation is missing or invalid. The LLM cannot lie about chart contents because it cannot bypass the structural gate.
"What about privacy/HIPAA?"	OAuth2 system-level auth, per-tool authorization scoped to the doctor's panel, append-only audit log of every chart access, BAA with LLM provider in production, demo data only in sprint.
"How does this scale to 300 concurrent doctors?"	FastAPI is async; FHIR calls parallelize; Anthropic prompt caching keeps per-turn cost flat; Postgres handles the audit log easily at this scale. Fly.io scales horizontally. Bottleneck would be OpenEMR itself, not us.
"Why LangGraph and not OpenAI SDK / DSPy / a custom loop?"	We need an explicit verification node between LLM and user. LangGraph's state-machine model makes that gate first-class; AgentExecutor hides it; raw SDK reinvents it.

Question	Answer
"What happens when the LLM is wrong?"	Three layers: (1) it physically can't cite a resource that doesn't exist (validator); (2) for medication safety it gets a second-pass fact-check; (3) doctor sees the citation and verifies it themselves — the right-side patient card makes this one click.
"What if a doctor asks about a patient they shouldn't see?"	Tool returns an error the LLM relays as "you don't have access to that record." The audit log captures the attempt.
"Why Fly.io?"	Fast deploy, managed Postgres, Docker-native (so the same image runs in AWS later), no vendor lock-in.